

Generative AI & Prompt Engineering

Study Notes — Neurofive Solutions Internship Track

Part 1: Generative AI Fundamentals

1.1 What Is Generative AI?

Generative AI refers to a class of artificial intelligence systems that create new content — text, images, audio, code, or video — rather than simply classifying or analyzing existing data. Unlike traditional predictive models, which output a label or a number, generative models output entirely new artifacts that resemble the patterns found in their training data.

The current generation of text-based generative AI is built on Large Language Models (LLMs), which are neural networks trained on massive text datasets to predict the next most likely word (or "token") in a sequence. Despite this relatively simple training objective, this next-token prediction approach, when scaled to billions of parameters, produces models capable of reasoning, summarizing, translating, and holding coherent conversations.

1.2 The Transformer Architecture

Modern LLMs are built on the Transformer architecture, introduced in a 2017 research paper titled "Attention Is All You Need." The key innovation of the Transformer is the self-attention mechanism, which allows the model to weigh the relevance of every other word in a sequence when processing a given word — regardless of how far apart they are in the text. This is a major improvement over earlier architectures like RNNs, which processed text sequentially and struggled to retain context over long passages.

Because self-attention can be computed in parallel across an entire sequence, Transformers train far more efficiently on modern hardware (GPUs and TPUs) than earlier sequential architectures, which is part of why LLMs have been able to scale to their current size.

1.3 Tokens and Context Windows

LLMs do not process text as individual characters or whole words — they process it as "tokens," which are sub-word units. A common rule of thumb is that 100 tokens is roughly equivalent to 75 English words. The "context window" of a model is the maximum number of tokens it can consider at once, including both the prompt and the response. A larger context window allows a model to reference more information at once — for example, an entire document instead of just a short excerpt — but very large context windows can also make it harder for a model to identify which specific detail is relevant to a given question, an effect sometimes called "needle in a haystack" difficulty.

1.4 Training Stages: Pretraining, Fine-Tuning, and RLHF

Most modern LLMs go through three broad training stages. First, **pretraining**, where the model learns general language patterns from a massive, largely unlabeled text corpus by predicting the next token. Second, **fine-tuning**, where the model is further trained on a smaller, curated dataset of high-quality

examples (often instruction-response pairs) to make it better at following instructions. Third, **RLHF** (Reinforcement Learning from Human Feedback), where human raters compare multiple model outputs and the model is optimized to produce responses humans prefer — improving helpfulness, honesty, and safety.

1.5 Reasoning Models vs. Standard Chat Models

A newer category of models, often called "reasoning models," are trained to generate an extended internal chain of reasoning before producing a final answer, rather than answering immediately. This tends to improve performance on tasks involving math, logic, and multi-step planning, though it typically comes with higher latency and computational cost per response. Standard chat models, by contrast, generate a response token by token without a distinct internal reasoning phase, which is why explicitly asking a standard chat model to "think step by step" (Chain-of-Thought prompting) can meaningfully improve its output on complex tasks — it approximates some of what reasoning models do by default.

Part 2: Prompt Engineering Techniques

2.1 What Is Prompt Engineering?

Prompt engineering is the practice of designing and refining the instructions given to an LLM in order to reliably get the output you want. Because LLMs generate responses based on patterns learned during training, the exact wording, structure, and context of a prompt can significantly change the quality, accuracy, and format of the response — even when the underlying model stays exactly the same. A well-engineered prompt typically does three things: gives the model enough context to understand the task, constrains the output format so the response is directly usable, and reduces ambiguity so the model doesn't have to guess what the user actually wants.

2.2 Zero-Shot Prompting

Zero-shot prompting means asking the model to complete a task with no examples provided — only an instruction. It works well for simple, well-defined tasks the model has likely seen many times during training, such as summarization or basic classification. It tends to perform worse on tasks with unusual formatting requirements or domain-specific edge cases, because the model has no reference point for what a correct answer should look like.

2.3 Few-Shot Prompting

Few-shot prompting provides the model with a small number of worked examples (typically 2 to 5) before asking it to complete a new, similar task. These examples act as a pattern the model can follow, which is especially useful when the desired output format is unusual, when the task requires distinguishing between subtle categories, or when consistency across many responses matters.

In an internal test on a 10-message customer support classification task (labeling each message as Complaint, Question, or Praise), a zero-shot prompt correctly classified 8 out of 10 messages on one model, while the same prompt converted to a few-shot format (with 3 labeled examples) correctly classified 10 out of 10 messages on every model tested. The two messages the zero-shot version got wrong were complaints phrased as questions (for example, "Why was I charged twice?"), which the model mistakenly classified based on surface sentence structure (the question mark) rather than the underlying intent (frustration).

2.4 Chain-of-Thought (CoT) Prompting

Chain-of-Thought prompting instructs the model to reason step by step before producing a final answer, rather than jumping straight to a conclusion. This is typically triggered with a phrase such as "think step by step before answering." CoT is particularly effective on tasks that involve arithmetic, logical reasoning, or multi-step decision-making, because it forces the model to lay out intermediate calculations that can be checked for errors, rather than presenting a final number with no visible work.

In one internal comparison on a business-decision math problem, both a plain prompt and a CoT + persona prompt reached the same correct final recommendation. However, the CoT + persona version produced a clearly labeled step-by-step breakdown (including ROI percentages and a side-by-side comparison table), while the plain prompt embedded the same math inline within prose. This shows

CoT's value isn't limited to fixing wrong answers — it also improves the clarity and auditability of correct ones.

2.5 Persona (Role) Prompting

Persona prompting means instructing the model to adopt a specific character, professional role, or point of view before responding. For example, a customer support chatbot might be given the system prompt: "You are FriendlyFix, a friendly and patient IT helpdesk assistant." This shapes not only the tone of responses, but also which rules the model consistently follows, such as redirecting off-topic questions back to the assistant's actual area of expertise rather than answering them directly.

In practice, a well-defined persona combined with explicit behavioral rules keeps a model in-character even when a user deliberately asks an unrelated question — for example, an IT helpdesk persona asked for a chicken curry recipe correctly declined and redirected the conversation back to IT topics, rather than attempting to answer.

2.6 Structured Outputs (JSON Mode)

Structured output prompting forces a model to respond in a fixed, machine-readable format — most commonly JSON — instead of free-form prose. This is essential when the model's output needs to be parsed directly by application code rather than read by a human. A robust structured output prompt should: define the exact schema expected (including field names and types), explicitly instruct the model to return only the JSON object with no surrounding explanation, and specify what should happen for fields with a fixed set of allowed values (an enum) when the input is ambiguous.

In one internal test extracting structured order data from restaurant messages, a deliberately ambiguous input caused a required enum field (`delivery_or_pickup`, which should only ever be "delivery" or "pickup") to come back as null — technically breaking the schema, even though the JSON itself was syntactically valid. Adding one explicit rule ("this field must always be delivery or pickup, never null — make your best guess from context") fixed the issue on re-test, changing the output from null to a correctly inferred value.

2.7 Prompt Constraints

Constraints are explicit rules placed on a prompt to control length, tone, format, or scope. Common constraint types include: length constraints ("respond in 3-4 sentences"), format constraints ("respond with only the category name, numbered 1-10"), tone constraints ("use a calm, reassuring tone"), and exclusion constraints ("do not mention pricing" or "do not promise a refund"). Constraints are especially important in production systems, since an unconstrained prompt may produce technically correct but unusably long, inconsistently formatted, or off-brand responses.

Part 3: Retrieval-Augmented Generation (RAG)

3.1 Why RAG Exists

LLMs only know what was present in their training data, which has a fixed cutoff date and does not include private or proprietary information such as a company's internal documents, a person's resume, or today's news. Retrieval-Augmented Generation (RAG) solves this by giving a model access to external documents at the time a question is asked, allowing it to answer using that specific source material instead of guessing from its training data. RAG is the core technique behind almost every "chat with your PDF" or "chat with your documents" tool.

3.2 The Four Stages of a RAG Pipeline

- **Chunking** — splitting a source document into smaller, manageable pieces of text, typically a few hundred tokens each, sometimes with overlap between chunks to avoid cutting a relevant idea in half.
- **Embedding** — converting each chunk into a numerical vector (a list of numbers, often several hundred to a few thousand dimensions) that represents its semantic meaning, using an embedding model.
- **Retrieval** — when a question is asked, converting the question into a vector the same way, then searching a vector database for the document chunks whose vectors are most mathematically similar to the question's vector.
- **Generation** — passing the retrieved chunks to the LLM along with the original question, instructing it to answer using only that grounded content rather than its general knowledge.

3.3 Why Not Just Paste the Whole Document Into the Prompt?

For short documents (a few pages), pasting the full text directly into a prompt can work reasonably well and skips the complexity of building a retrieval pipeline. RAG becomes necessary once a document (or a collection of documents) is larger than the model's context window, or when repeatedly re-sending a large document for every single question would be too slow or too expensive. RAG's main advantage is that it retrieves only the small number of chunks relevant to each specific question, rather than sending the entire source material every time.

3.4 Common Tools for Building RAG Systems

Coding frameworks such as LangChain and LlamaIndex provide pre-built components for chunking, embedding, vector storage, and retrieval, making it faster to assemble a custom RAG pipeline in Python. For teams or individuals who want RAG-like functionality without writing code, no-code tools such as Google's NotebookLM, Claude Projects, and custom GPTs with file upload provide a similar experience: upload a document, then ask questions that are answered using that document's content.

3.5 Hallucination and Grounding

A hallucination occurs when a model generates information that sounds plausible and confident but is not actually supported by its source data — whether that source is its training data, or, in a RAG

system, the retrieved document chunks. Grounding a model's answers in a specific document significantly reduces hallucination compared to a plain prompt with no source material, because the model has concrete text to reference rather than needing to generate an answer purely from memory.

However, grounding does not eliminate hallucination entirely. A model can still misread or over-generalize from a retrieved chunk, especially when a question's true answer is only partially present in the document, requires combining information from multiple sections, or when the retrieval step itself fails to surface the most relevant chunk in the first place.

Part 4: Evaluating & Using Generative AI Responsibly

4.1 Evaluating Prompt Quality

A prompt should be evaluated the same way any other engineering decision is: by testing it against multiple representative inputs, not just one. A prompt that works on a single example may fail on edge cases — ambiguous phrasing, missing information, or unusually structured input. A reliable evaluation process typically includes: testing on at least 5 varied sample inputs, deliberately trying to "break" the prompt with a messy or adversarial input, and iterating on the prompt based on what specifically went wrong, rather than rewriting it from scratch.

4.2 API Response Parsing

When an LLM is accessed programmatically through an API rather than a chat interface, its response must be parsed into a format the surrounding application code can use. For structured (JSON) outputs, this typically means validating that the response is syntactically valid JSON before attempting to read individual fields, since even a well-designed prompt can occasionally produce a malformed response that would crash a program expecting a specific structure.

4.3 API Keys and Credential Safety

When connecting to an LLM API from code, the API key should never be hardcoded directly into a script that will be shared, committed to a public repository, or shown on screen (including in screen recordings). The standard practice is to store the key in a separate environment file (such as a `.env` file), load it into the program at runtime using a library such as `python-dotenv`, and exclude that file from version control using a `.gitignore` entry. A placeholder file, often named `.env.example`, can be committed instead, showing collaborators which environment variable name is required without exposing any real credentials.

4.4 Rate Limits and Free-Tier Quotas

Most LLM API providers enforce rate limits on free-tier accounts, typically expressed as a maximum number of requests per minute (RPM) and/or per day (RPD). Exceeding this limit returns a "resource exhausted" or "quota exceeded" error rather than a normal response. When testing a script that makes several API calls in a row (for example, running 5 test messages through a chatbot), adding a short delay between calls reduces the risk of hitting these limits.

4.5 Key Takeaways

- Generative AI models create new content by predicting likely next tokens, built on the Transformer architecture and its self-attention mechanism.
- Prompt wording alone can change accuracy substantially — in one internal test, few-shot prompting improved a classification task from 80% to 100% accuracy on a model that struggled zero-shot.
- Chain-of-Thought prompting improves reasoning transparency and auditability, even when the final answer would have been correct without it.

- Persona prompting is most effective when paired with explicit behavioral rules, not just a role label.
- Structured output prompts must explicitly define behavior for ambiguous or missing values, especially for fixed-choice (enum) fields.
- RAG reduces — but does not eliminate — hallucination, and works through four stages: chunking, embedding, retrieval, and generation.
- API credentials should always be kept out of source code and screen recordings, typically using a `.env` file excluded from version control.